Bellhaven CRM Pipeline

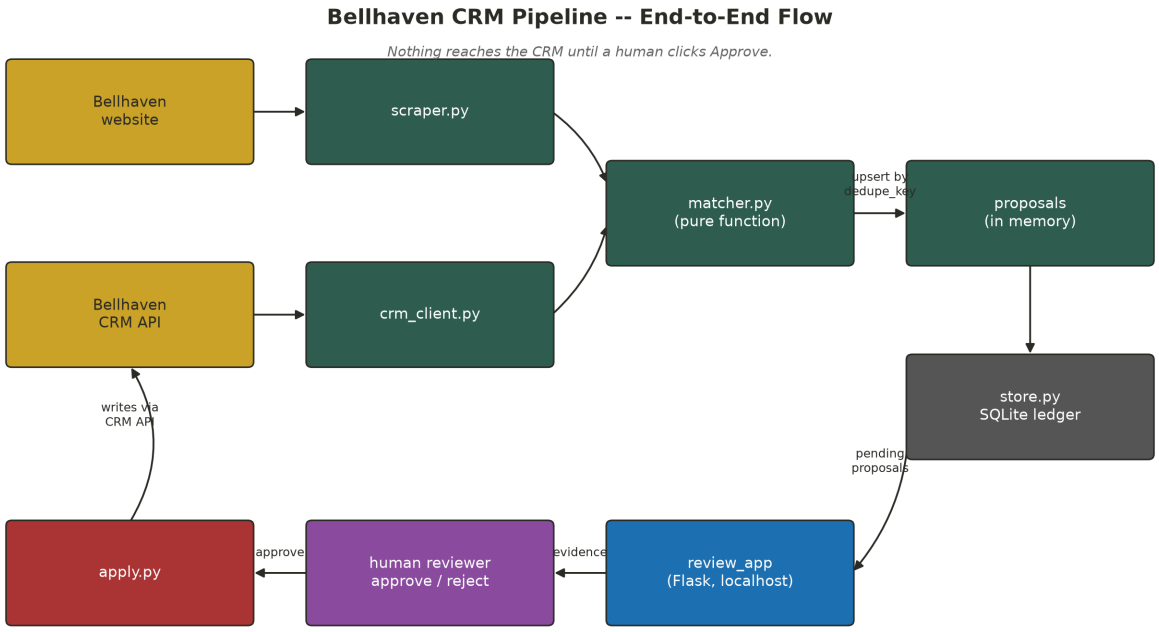
Daily scrape -> match -> review -> write-back pipeline for the Bellhaven Senior Living CRM sandbox

github.com/dparikh066-stack/bellhaven-crm-pipeline

The Problem

- ~60% of the LTC facilities in this CRM are owned by a parent company, and ownership changes constantly -- acquired, rebranded, split, consolidated.
- Every ownership event can quietly break the parent_id link between a facility and its parent account.
- Corporate-owned vs. independent facilities are sold to differently -- an inaccurate ownership picture is a real, ongoing operational problem.
- Goal: keep the CRM's picture of which facilities Bellhaven owns accurate, automatically, every day -- without ever writing to the CRM unreviewed.

Architecture Overview



- scraper.py and crm_client.py independently pull the two sources of truth: the public website and the CRM API.
- matcher.py is a pure function -- (locations, accounts) -> proposals -- with no I/O, so it's trivial to reason about and re-run.
- store.py is a SQLite ledger that turns repeated runs into an idempotent upsert instead of duplicate proposals.
- review_app is the only thing allowed to write to the CRM, and only after a human clicks Approve.

Component: The Scraper

- Walks /communities?page=N and parses every listing card: name, street, city, state, zip, and one or more care-offering badges.
- Also scans the homepage for /communities/{slug} links -- a just-announced acquisition (Bellhaven Meadows of Findlay) was linked from a promo banner before it ever made it into the paginated directory grid.
- Result: 35 locations captured, not the 34 the paginated grid alone would have found.
- Output is a plain list of dicts (data/locations.json) -- no CRM knowledge lives in this module.

Component: Matching Signals

- Every scraped location is scored against every CRM account on street similarity, name similarity, and exact zip/city/state matches.
- Two independent confirmation tests sit on top of the raw score, because the data breaks in two different ways:
- Address-confirmed: zip matches and the street reads the same ("875 Elm Street" vs "875 Elm St").
- Identity-confirmed: zip+city+name are strong even when the street is stale -- a PO box on file (Ashtabula), or the zip itself is wrong, a transposed digit (Portsmouth: 45626 vs 45662).

Code: Scoring & Confirmation

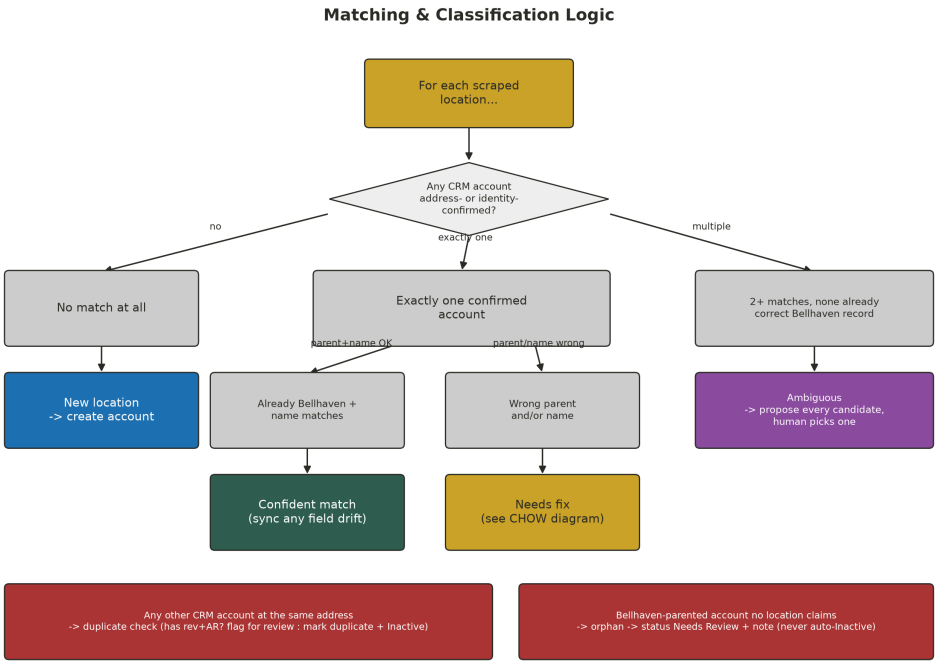
```
def score_pair(loc, acct):
    a_street = norm_street(acct.get("billing_street"))
    a_name = norm_name(acct.get("name"))
    street_sim = difflib.SequenceMatcher(None, norm_street(loc["street"]), a_street).ratio()
    name_sim = difflib.SequenceMatcher(None, norm_name(loc["name"]), a_name).ratio()
    zip_match = loc["zip"] == acct.get("billing_zip")
    a_city = (acct.get("billing_city") or "").strip().lower()
    city_match = loc["city"].strip().lower() == a_city
    state_match = loc["state"] == acct.get("billing_state")
    combined = (0.4 * street_sim + 0.25 * name_sim + 0.2 * zip_match
                + 0.1 * city_match + 0.05 * state_match)
    return {"street_sim": street_sim, "name_sim": name_sim,
            "zip_match": zip_match, "combined": combined}


def is_address_confirmed(s):
    """Same physical address: zip matches and the street reads the same."""
    return s["zip_match"] and s["street_sim"] >= 0.75


def is_identity_confirmed(s):
    """Same facility even if one field is stale (a PO-box billing street,
    or a transposed zip digit) -- name/city/state carry enough signal alone."""
    if s["zip_match"] and s["city_match"] and s["name_sim"] >= 0.85:
        return True
    if (s["name_sim"] >= 0.95 and s["street_sim"] >= 0.85
            and s["city_match"] and s["state_match"]):
        return True
    return False
```

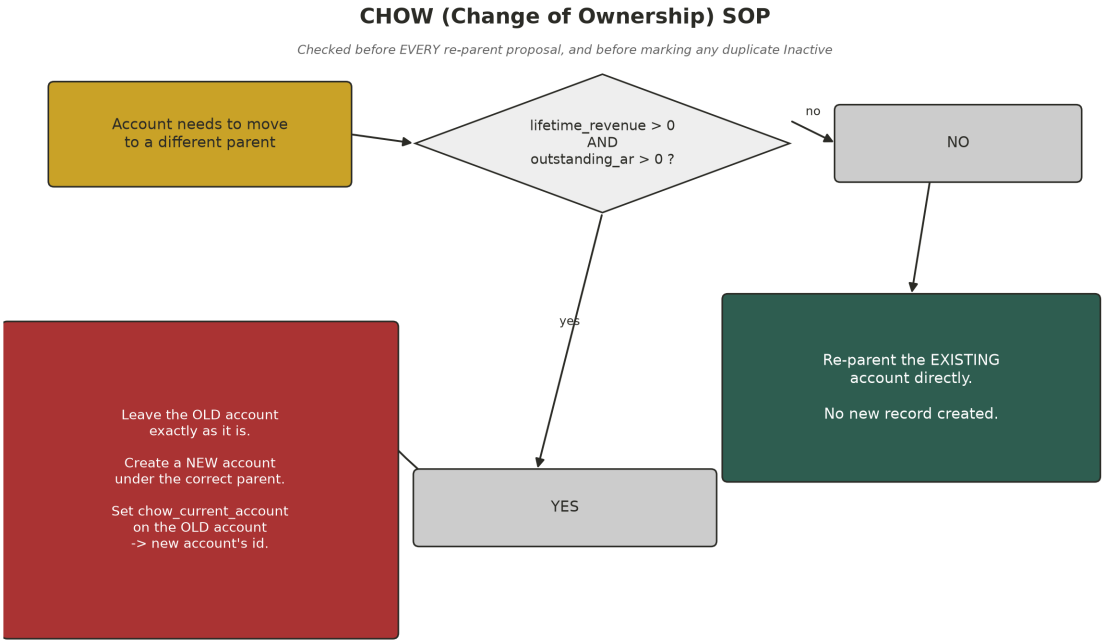
- score_pair() is pure and side-effect free -- same inputs always produce the same score, which is what lets matcher.build_proposals() be a pure function end to end.
- Two confirmation paths, not one threshold: is_address_confirmed catches reformatted streets; is_identity_confirmed catches a stale/wrong street when name+city+zip still agree.
- Portsmouth needed a third path in practice: name+street+city+state strong even when zip itself was the wrong field (a transposed digit) -- handled in the full identity check in matcher.py.

Classification Logic



- Confident match -- right account, already correctly parented and named. Still checked field-by-field for drift (stale phone numbers turned out to be extremely common).
- Needs fix -- right account exists, but the parent and/or name is wrong.
- New location -- nothing matches; propose creating the account.
- Ambiguous -- 2+ CRM accounts at the same address, none already the correct Bellhaven record. No auto-pick: every candidate is proposed as a mutually exclusive alternative for a human to choose.
- Orphan -- a Bellhaven-parented account no scraped location claims. Always Needs Review, never auto-Inactive.

The CHOW (Change of Ownership) SOP



- Implemented literally: before re-parenting an existing account, check lifetime_revenue and outstanding_ar.
- Both > 0 -> leave the old account completely untouched (only chow_current_account gets set), create a new account under the correct parent, point the old one at the new one.
- Otherwise -> re-parent the existing account directly, no new record.
- Fired for real on two accounts: Marietta and Tiffin, both under Cedar Trail Communities with real billing history.
- Second-order effect: after a split, old and new accounts share an address, so duplicate-detection finds the old one again -- correctly routes it to human review instead of auto-deactivating it, since it still carries revenue + AR.

Code: The CHOW Check

```
def needs_chow_split(account):
    """CHOW SOP: preserve the old account (don't reparent it) only when it
    has BOTH revenue history AND currently-outstanding AR."""
    return (account.get("lifetime_revenue") or 0) > 0 \
       and (account.get("outstanding_ar") or 0) > 0

# ... in build_proposals(), whenever a match's parent is wrong:
if needs_chow_split(anchor_acct):
    # leave anchor_acct's parent/status untouched; propose a NEW account
    # under Bellhaven, and patch anchor_acct.chow_current_account -> new id
    ...
else:
    # no revenue/AR entanglement -- safe to re-parent anchor_acct directly
    ...
```

- One boolean gate, checked in exactly one place, applied consistently everywhere a parent could change -- including the ambiguous-Kettering candidates, had one been approved with revenue on the books.
- "Preserve" means literally not included in that proposal's proposed_changes at all -- the old account's PATCH call only ever sets chow_current_account, nothing else.

Data Quality Issues Found & Handled

Issue	Example	Resolution
Stale phone numbers	~13 accounts	synced from website
Reformatted/abbreviated street	Toledo, Marion, Ashtabula...	synced from website
Stale billing address (PO box)	Ashtabula	synced physical address
Transposed zip digit	Portsmouth (45626->45662)	corrected
Outdated name after rebrand	Chesterton, Chagrin Falls, Willow Creek	renamed to match website
Wrong parent, no billing history	Findlay, Lima	re-parented directly
Wrong parent, real revenue+AR	Marietta, Tiffin	CHOW split
Exact duplicate, same address	Owosso (x2), Port Clinton, Erie, Monroe	duplicate_of_account + Inactive
Multiple same-address, no survivor	Kettering (3 pre-acquisition records)	left pending for a human
Active in CRM, gone from website	Alliance, Coldwater, Sandusky	flagged Needs Review, never auto-closed
New community, no CRM account	Batavia, Union Square, Carlisle, Amberly Manor	created
Name collision, unrelated facility	"Amberly Manor" also exists in Colorado Springs	correctly not matched -- different state

Making the Note Field Do Its Job

- The assignment names status AND the free-text note field as the two tools for expressing findings that aren't simple field updates.
- Duplicate and orphan proposals write an explanation into the CRM's actual note field, not just the review app's evidence panel -- status alone doesn't say WHY an account was deactivated or flagged.
- Caught this gap during a self-review after the first full run, fixed the matcher, and backfilled the 8 accounts already approved before the fix so the live CRM matches what the corrected pipeline produces.

Code: Idempotent Re-Runs

```
def dedupe_key_for(p):
    ct = p["change_type"]
    if ct == "chow_split":
        return f"chow:{slug}:{ids}"
    if ct == "mark_duplicate":
        return f"duplicate:{ids}:{p['proposed_changes']['duplicate_of_account']}"
    if ct == "flag_review" and p["classification"] == "orphan":
        return f"orphan:{ids}"
    ...

# upsert_proposals(): a proposal whose dedupe_key already exists with
# status in {approved, rejected} is NEVER modified again -- only a
# 'pending' row gets its content refreshed, or gets auto_resolved if the
# matcher stops producing it on a later run.
```

- Every proposal type has its own deterministic key format -- based on the location slug and/or account id(s), never on anything that changes run to run.
- This is the entire idempotency mechanism: no separate "already processed" flag, no timestamps compared -- just an upsert keyed on content identity.

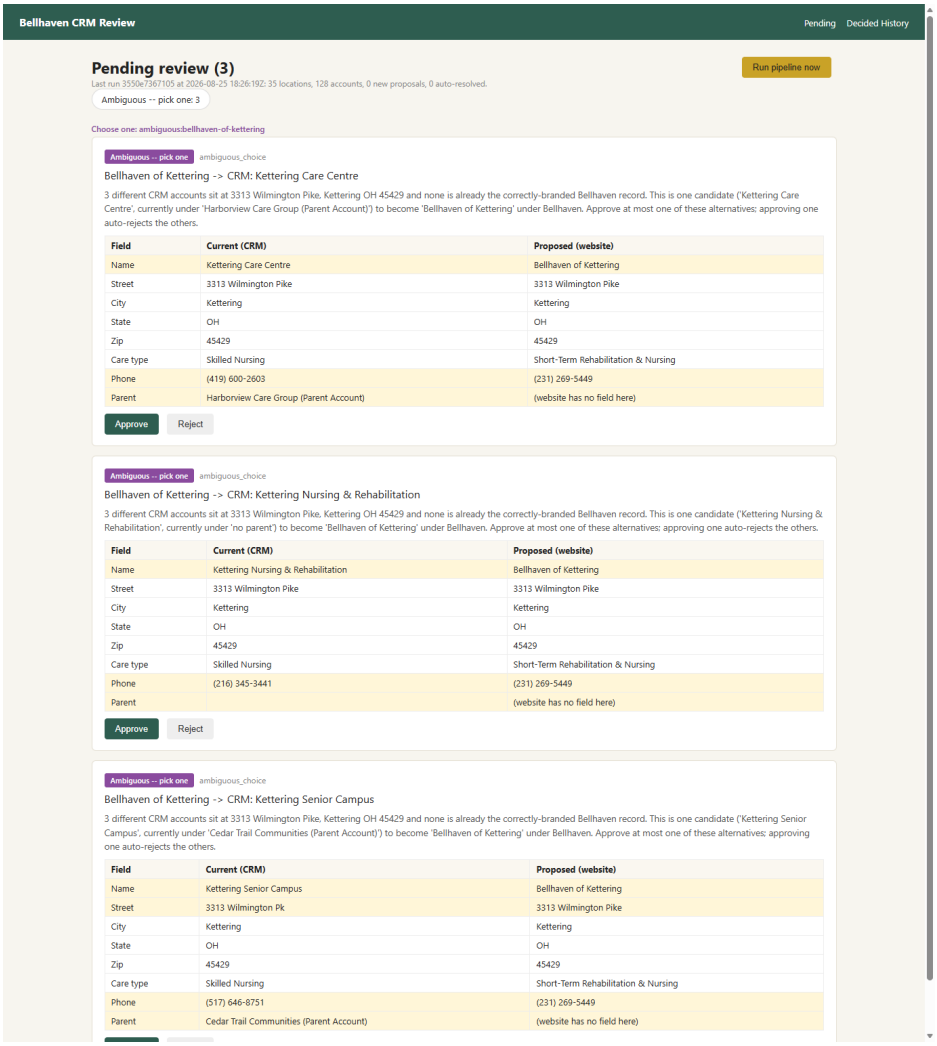
Idempotency, Verified Live

- matcher.build_proposals is a pure function of current CRM + website state -- every run regenerates the entire proposal set from scratch.
- A proposal that already has a decision (approved/rejected) is never touched again -- only its last_seen_run_id is bumped.
- A pending proposal the matcher no longer produces gets auto-closed as auto_resolved (someone fixed it by hand, or an approval already resolved it).
- Verified live: re-running after approving 33 proposals produced 0 duplicate re-proposals of anything already decided -- only 2 genuinely new items surfaced (the expected post-CHOW same-address flags, described above).

The Review App

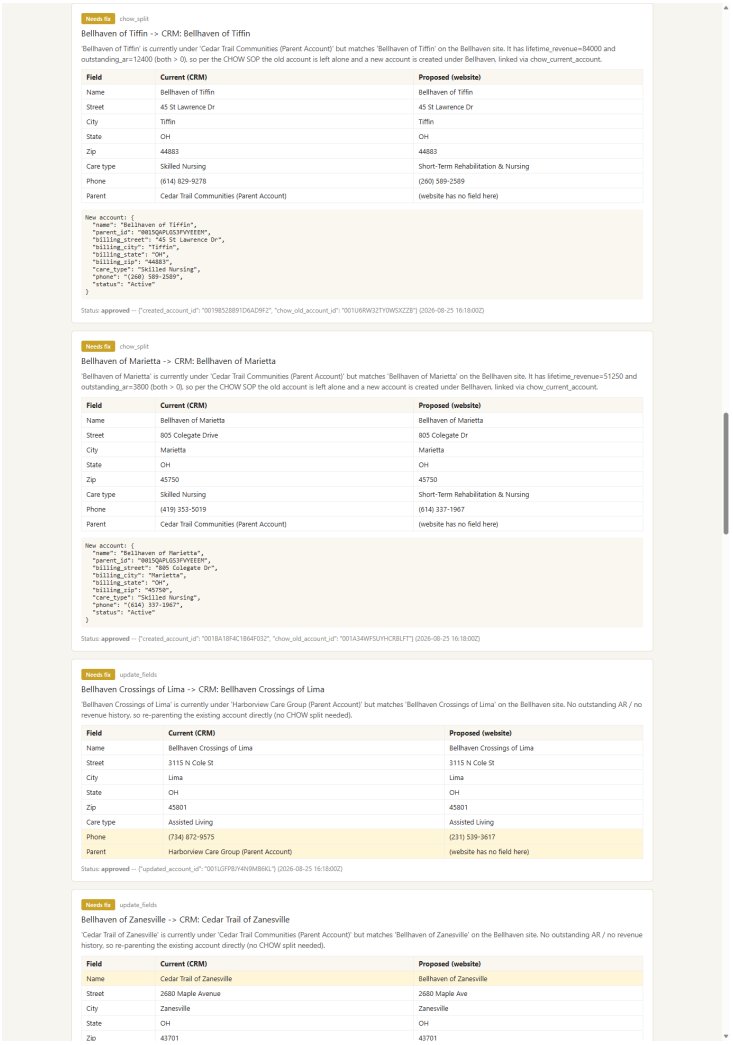
- Local Flask app (python -m review_app.app, http://127.0.0.1:5000) -- the only place a CRM write can originate from.
- Every proposal card shows: classification, the plain-language rationale, a current-vs-proposed field diff table, and (for creates/CHOW) the full new-account payload.
- Approve calls the real CRM API immediately; Reject just records the decision. Nothing else writes to the CRM.
- Ambiguous groups (like Kettering) are rendered together; approving one automatically rejects its siblings.
- A later addition: Reopen for review on rejected proposals -- undoes a reject decision without needing a full re-run (deliberately blocked for approved proposals, since those already made a real, un-undoable CRM write).

Screenshot: Pending Review Queue



- The 3 Kettering alternatives, live in the app -- rendered as a "Choose one" group.
- Each card shows the full current-vs-proposed field diff, plus which parent/account it would come from.
- Approving any one of the three automatically rejects the other two -- enforced by the app, not left to the reviewer to remember.

Screenshot: A Real CHOW Split, Approved



Needs fix chow_split

Bellhaven of Tiffin -> CRM: Bellhaven of Tiffin

'Bellhaven of Tiffin' is currently under 'Cedar Trail Communities (Parent Account)' but matches 'Bellhaven of Tiffin' on the Bellhaven site. It has lifetime_revenue=84000 and outstanding_ar=12400 (both > 0), so per the CHOW SOP the old account is left alone and a new account is created under Bellhaven, linked via chow_current_account.

Field	Current (CRM)	Proposed (website)
Name	Bellhaven of Tiffin	Bellhaven of Tiffin
Street	45 St Lawrence Dr	45 St Lawrence Dr
City	Tiffin	Tiffin
State	OH	OH
Zip	44883	44883
Care type	Skilled Nursing	Short-Term Rehabilitation & Nursing
Phone	(614) 829-9278	(260) 589-2589
Parent	Cedar Trail Communities (Parent Account)	(website has no field here)


```
New account: {
  "name": "Bellhaven of Tiffin",
  "parent_id": "001SQAPLGS3FVYEEEM",
  "billing_street": "45 St Lawrence Dr",
  "billing_city": "Tiffin",
  "billing_state": "OH",
  "billing_zip": "44883",
  "care_type": "Skilled Nursing",
  "phone": "(260) 589-2589",
  "status": "Active"
}
```

Status: **approved** -- {"created_account_id": "0019852B891D6AD9F2", "chow_old_account_id": "001U6RW32TY0W5XZZB"} (2026-08-25 16:18:00Z)

Needs fix chow_split

Bellhaven of Marietta -> CRM: Bellhaven of Marietta

'Bellhaven of Marietta' is currently under 'Cedar Trail Communities (Parent Account)' but matches 'Bellhaven of Marietta' on the Bellhaven site. It has lifetime_revenue=51250 and outstanding_ar=3800 (both > 0), so per the CHOW SOP the old account is left alone and a new account is created under Bellhaven, linked via chow_current_account.

Field	Current (CRM)	Proposed (website)
Name	Bellhaven of Marietta	Bellhaven of Marietta
Street	805 Colegate Drive	805 Colegate Dr
City	Marietta	Marietta
State	OH	OH
Zip	45750	45750
Care type	Skilled Nursing	Short-Term Rehabilitation & Nursing
Phone	(419) 353-5019	(614) 337-1967
Parent	Cedar Trail Communities (Parent Account)	(website has no field here)


```
New account: {
  "name": "Bellhaven of Marietta",
  "parent_id": "001SQAPLGS3FVYEEEM",
  "billing_street": "805 Colegate Dr",
  "billing_city": "Marietta",
  "billing_state": "OH",
  "billing_zip": "45750",
  "care_type": "Skilled Nursing",
  "phone": "(614) 337-1967",
  "status": "Active"
}
```

Status: **approved** -- {"created_account_id": "001BA18F4C1864F032", "chow_old_account_id": "001A34WF5UYHC8LFT"} (2026-08-25 16:18:00Z)

Needs fix update_fields

Bellhaven Crossings of Lima -> CRM: Bellhaven Crossings of Lima

'Bellhaven Crossings of Lima' is currently under 'Harborview Care Group (Parent Account)' but matches 'Bellhaven Crossings of Lima' on the Bellhaven site. No outstanding AR / no revenue history, so re-parenting the existing account directly (no CHOW split needed).

Field	Current (CRM)	Proposed (website)
Name	Bellhaven Crossings of Lima	Bellhaven Crossings of Lima
Street	3115 N Cole St	3115 N Cole St
City	Lima	Lima
State	OH	OH
Zip	45801	45801
Care type	Assisted Living	Assisted Living
Phone	(734) 872-9575	(231) 539-3617
Parent	Harborview Care Group (Parent Account)	(website has no field here)

Status: **approved** -- {"updated_account_id": "001LGFPB/Y4N9MB6KL"} (2026-08-25 16:18:00Z)

Needs fix update_fields

Bellhaven of Zanesville -> CRM: Cedar Trail of Zanesville

'Cedar Trail of Zanesville' is currently under 'Cedar Trail Communities (Parent Account)' but matches 'Bellhaven of Zanesville' on the Bellhaven site. No outstanding AR / no revenue history, so re-parenting the existing account directly (no CHOW split needed).

Field	Current (CRM)	Proposed (website)
Name	Cedar Trail of Zanesville	Bellhaven of Zanesville
Street	2680 Maple Avenue	2680 Maple Ave
City	Zanesville	Zanesville
State	OH	OH
Zip	43701	43701

- Decided History for Bellhaven of Tiffin: rationale, full diff table, the exact new-account JSON payload that was POSTed, and the applied result -- created_account_id + chow_old_account_id -- all in one card.
- Directly below it, Lima's direct re-parent (no CHOW needed) shows the contrast: same 'Needs fix' classification, different resolution, because outstanding_ar was 0.

Built to Run Daily

- schedule/crontab.example -- a plain cron entry running pipeline.py once a day.
- .github/workflows/daily-pipeline.yml -- equivalent GitHub Actions schedule (cron trigger + manual workflow_dispatch).
- Neither is live, per the assignment -- both are ready to enable.
- The GitHub Actions version commits data/pipeline.db back to the repo after each run, so the idempotency ledger survives between ephemeral runner instances.

Results: What Actually Landed in the CRM

- 35 locations scraped, 121 CRM accounts at the start -- 36 proposals generated on the first run.
- 33 approved and applied through the real review app: 13 field syncs, 6 renames/re-parents, 2 CHOW splits, 5 duplicates marked Inactive, 3 orphans flagged Needs Review, 4 new accounts created.
- 2 more surfaced on the very next run and were rejected: the expected post-CHOW same-address flags on the Tiffin/Marietta OLD accounts -- correctly left untouched per the SOP.
- 3 left pending on purpose: the Kettering ambiguous-alternatives group -- see below.
- Every CHOW split, duplicate, and orphan outcome was individually verified against the live CRM API after approval, not assumed from the proposal text.

Full Change Log (1/2)

#	Location / Account	Classification	What Happened
1	Bellhaven of Goshen	Confident match	Updated: phone
2	Bellhaven of Defiance	Confident match	Updated: phone
3	Bellhaven Woods of Toledo	Confident match	Updated: billing_street
4	Bellhaven of Marion	Confident match	Updated: phone
5	Bellhaven Rehab & Nursing of Grove City	Confident match	Updated: name
6	Bellhaven at Sycamore Ridge	Confident match	Updated: phone
7	The Arbors at Bellhaven - Dayton	Confident match	Updated: phone
8	Bellhaven Healthcare Centre of Ashland	Confident match	Updated: name, phone
9	Bellhaven of Port Clinton	Confident match	Updated: phone
10	Harborview Nursing & Rehab of Port Clinton	Duplicate	duplicate_of_account -> Port Clinton, Inactive
11	Bellhaven Shores of Erie	Confident match	Updated: phone
12	Harborview Shores of Erie	Duplicate	duplicate_of_account -> Bellhaven Shores of Erie, Inactive
16	Bellhaven Gardens of Monroe	Confident match	Updated: phone
17	Monroe Gardens Care Center	Duplicate	duplicate_of_account -> Gardens of Monroe, Inactive
18	Cedar Trail of Monroe	Duplicate	duplicate_of_account -> Gardens of Monroe, Inactive
19	Bellhaven of Tiffin	Needs fix	CHOW split -- old preserved, new account created
20	Bellhaven of Marietta	Needs fix	CHOW split -- old preserved, new account created
21	Bellhaven Crossings of Lima	Needs fix	Updated: parent_id, phone (direct re-parent)

Full Change Log (2/2)

#	Location / Account	Classification	What Happened
22	Bellhaven of Zanesville	Needs fix	Updated: parent_id, name, phone
23	Bellhaven of Chagrin Falls	Needs fix	Updated: name, phone (rename only)
24	Bellhaven Willow Creek	Needs fix	Updated: name, phone (rename only)
25	Bellhaven of Ashtabula	Confident match	Updated: billing_street (PO box -> physical)
26	Bellhaven of Owosso (dup)	Duplicate	duplicate_of_account -> Owosso, Inactive
27	Bellhaven of Portsmouth	Confident match	Updated: billing_zip (typo fix)
28	Bellhaven of Batavia	New location	Created under Bellhaven
29	Bellhaven at Union Square	New location	Created under Bellhaven
30	Bellhaven of Carlisle	New location	Created under Bellhaven
31	Amberly Manor (Hudson, OH)	New location	Created under Bellhaven
32	Bellhaven of Chesterton	Needs fix	Updated: name, billing_street
33	Bellhaven Meadows of Findlay	Needs fix	Updated: parent_id, phone (direct re-parent)
34	Bellhaven Care Center of Alliance	Orphan	status -> Needs Review + note
35	Bellhaven of Coldwater	Orphan	status -> Needs Review + note
36	Bellhaven of Sandusky	Orphan	status -> Needs Review + note (has $130k rev / $5.2k AR)
37	Bellhaven of Tiffin (OLD, post-CHOW)	Rejected	flag_review REJECTED -- old account left untouched, per SOP
38	Bellhaven of Marietta (OLD, post-CHOW)	Rejected	flag_review REJECTED -- old account left untouched, per SOP

Kettering: Left Unresolved, Deliberately

- Three different CRM accounts sit at 3313 Wilmington Pike -- "Kettering Care Centre" (Harborview), "Kettering Nursing & Rehabilitation" (unparented), "Kettering Senior Campus" (Cedar Trail).
- None is already correctly named or parented for Bellhaven. All three have zero revenue and zero AR -- no billing signal to break the tie.
- Nothing in the scraped data indicates which pre-acquisition entity is meant to continue as "Bellhaven of Kettering."
- Auto-picking one risked silently misattributing history to the wrong legal entity, so the pipeline proposes all three as mutually exclusive alternatives and leaves the decision pending for a person with more context (e.g. corporate development).

What I'd Do Differently With More Time

- The fuzzy-matching thresholds in matcher.py are hand-tuned against this dataset -- a production version would want a labeled match/no-match set to validate against.
- care_type is single-select in the CRM but the website lists multiple care offerings for a few communities (e.g. Erie) -- currently maps to the first offering only.
- The SQLite ledger works at this scale (~120 accounts); at real scale, a hosted DB would replace the git-commit-as-database approach used in the GitHub Actions workflow.

Repository

- github.com/dparikh066-stack/bellhaven-crm-pipeline
- scraper.py, crm_client.py, matcher.py, store.py, pipeline.py, apply.py, review_app/
- schedule/crontab.example, .github/workflows/daily-pipeline.yml
- README.md (setup + quick start), WRITEUP.md (full design rationale)